**Name: Miheeka Kewalramani Roll No.45 Branch:AI&DS Class:T12 Subject:ASD&D**

## Experiment 05

**Aim:** To automate application deployment using GitHub and Jenkins.

This experiment maps to LO3

**LO3**: Build and manage automated CI/CD pipelines for application development, testing, packaging, and deployment.

**Theory:**

Continuous Integration (CI) and Continuous Deployment (CD) are important DevOps practices that help automate and streamline the software development and delivery process.

- **Continuous Integration (CI):** It is a software development practice in which developers frequently integrate their code changes into a shared repository. The changes are automatically built and tested to identify errors at an early stage, improve software quality, and reduce integration issues.
- **Continuous Deployment (CD):** It is the automated process of delivering tested and verified code changes to staging or production environments. This allows new features and fixes to be released quickly with minimal manual effort.
- **Jenkins & Declarative Pipelines:** Jenkins is an open-source automation tool used to automate building, testing, and deployment tasks. A **Declarative Pipeline** uses a structured Groovy-based syntax to define stages, steps, and execution environments in a **Jenkinsfile**. This makes the process easier to manage, reproduce, and maintain using version control.

**Step-by-Step Procedure**

**Step 1: Jenkins Server Initialization and Access Setup**

- Jenkins runs as a background Windows service requiring a compatible Java Development Kit (JDK 21) runtime environment to execute its core processes.
- Access the web console at http://localhost:8080 to complete the setup wizard.
- Create the first administrator user to manage permissions, job configurations, and server tasks.

**Step 2: Baseline Pipeline Validation (Inline Scripting)**

- Create a new project item named GitHub-Jenkins-Demo using the native **Pipeline** job type.
- Add an inline Declarative Pipeline script directly to the job configuration to test the server's execution engine prior to external version control integration.

- Use the agent any directive along with three core lifecycle stages: **Build**, **Test**, and **Deploy** (each executing standard echo statements).
- Trigger **Build #1**; verify via the console logs that Jenkins assigns a local node executor in the workspace directory, processes each stage sequentially, and finishes with SUCCESS.

**Step 3: Source Code Management (SCM) & Pipeline-as-Code Implementation**

- Establish a central GitHub repository named jenkins-demo-app under the user profile to maintain the application codebase and configuration.
- Add an index.html file as the frontend application artifact with an initial header message.
- Define a version-controlled Jenkinsfile in the repository root directory using Declarative Pipeline syntax.
- Script automated batch instructions (bat) within the **Deploy** stage to verify the destination directory (C:\Deployments\DemoApp) and copy index.html from the workspace to the target folder.

**Step 4: Connecting Jenkins to GitHub and Initial Deployment**

- Update the Jenkins project configuration from an inline script to **Pipeline script from SCM**, specifying **Git** as the source provider.
- Configure the repository HTTPS URL, set the branch specifier to */main, and target the repository's Jenkinsfile.
- Run **Build #2** to trigger Jenkins to clone the remote repository, checkout the latest commit, parse the Jenkinsfile, and execute the deployment batch commands.
- Verify deployment by opening C:/Deployments/DemoApp/index.html in a web browser.

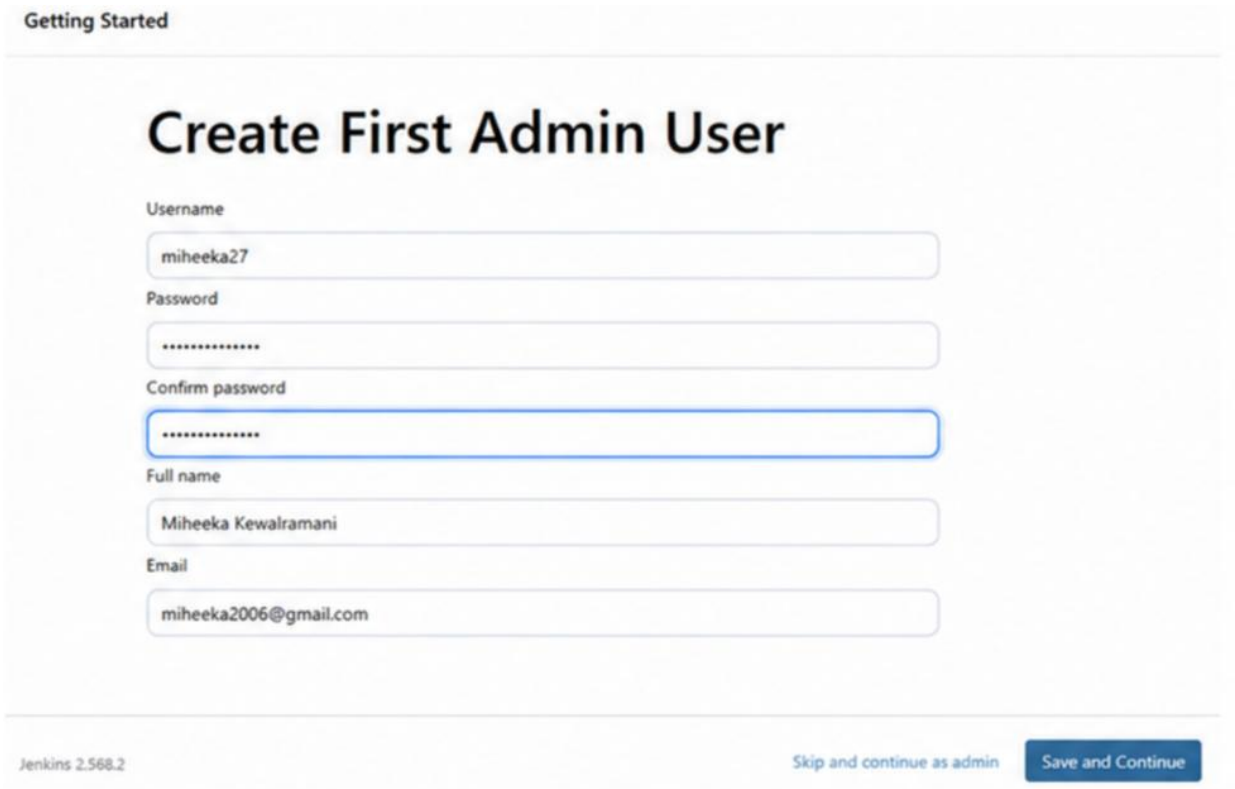
**Step 5: Continuous Integration & Continuous Deployment (CI/CD) Verification**

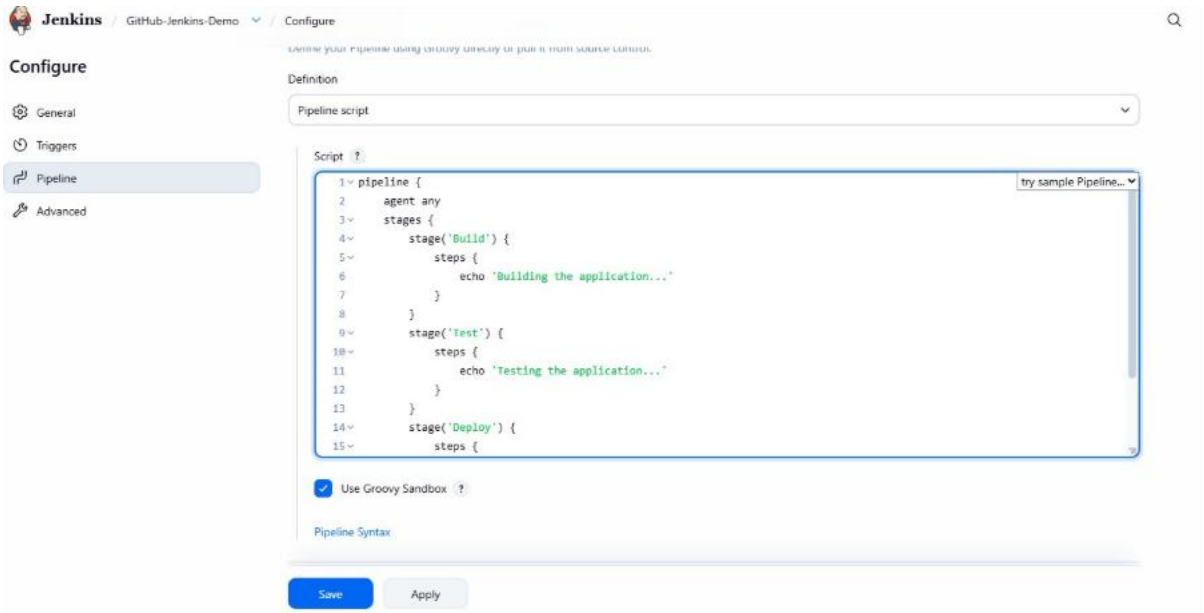
- Modify index.html directly on GitHub by adding custom CSS styling and updating the text to:

Application Updated & Deployed via CI/CD Pipeline! Version 2.0 is live.

- Trigger **Build #3**; Jenkins fetches the updated commit hash, executes all stages, and overwrites the existing index.html in C:\Deployments\DemoApp.
- Refresh the browser page to confirm the updated UI and Version 2.0 text, validating the end-to-end automated delivery pipeline.

**Execution: -**


Getting Started
Create First Admin User
Username
miheeka27
Password
Confirm password
Full name
Miheeka Kewalramani
Email
miheeka2006@gmail.com
Jenkins 2.568.2
Skip and continue as admin
Save and Continue


Jenkins
GitHub-Jenkins-Demo
Configure
Configure
General
Triggers
Pipeline
Advanced
Definition
Pipeline script
Script
try sample Pipeline...
pipeline {
agent any
stages {
stage('Build') {
steps {
echo 'Building the application...'
}
}
stage('Test') {
steps {
echo 'Testing the application...'
}
}
stage('Deploy') {
steps {
Use Groovy Sandbox
Pipeline Syntax
Save
Apply

## New Item

Enter an item name

GitHub-Jenkins-Demo

Select an item type

**Pipeline**
Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.

**Freestyle project**
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.

**Multi-configuration project**
Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.

**Folder**
Creates a container that stores nested items in it. Useful for grouping things together. Unlike view, which is just a filter, a folder creates a separate namespace, so you can have multiple things of the same name as long as they are in different folders.

**Multibranch Pipeline**
Creates a set of Pipeline projects according to detected branches in one SCM repository.

OK

GitHub-Jenkins-Demo / #1

atus
anges '1'
uild Overview
age View
uild Information
peline Stage

## Console

```
Started by user Miheeka Kewalramani
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in C:\ProgramData\Jenkins\.jenkins\workspace\GitHub-Jenkins-Demo
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Build)
[Pipeline] echo
Building the application...
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Test)
[Pipeline] echo
Testing the application...
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Deploy)
[Pipeline] echo
Deploying the application...
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

jenkins-demo-app / Jenkinsfile

ascendantcreator Rename Jenkinsfile.txt to Jenkinsfile 8c8d21c 9 mi

Code Blame 22 lines (22 loc) · 578 Bytes Raw

```
pipeline {
    agent any
    stages {
        stage('Build') {
            steps {
                echo 'Building the application...'
            }
        }
        stage('Test') {
            steps {
                echo 'Testing the application...'
            }
        }
        stage('Deploy') {
            steps {
                echo 'Deploying the application...'
                bat 'If not exist "C:\\Deployments\\DemoApp" mkdir "C:\\Deployments\\DemoApp"'
                bat 'copy index.html "C:\\Deployments\\DemoApp\\Index.html"'
            }
        }
    }
}
```

jenkins-demo-app / index.html in main Cancel changes Commit changes...

Edit Preview Spaces 4 No wrap

```
<!DOCTYPE html>
<html>
<head>
    <title>CI/CD Demo Application</title>
</head>
<body>
    <h1>Application Deployed Successfully via Jenkins!</h1>
</body>
</html>
```

Define your Pipeline using Groovy directly or pull it from source control.

Definition

Pipeline script from SCM

SCM ?

Git ?

Repositories ?

Repository URL ?

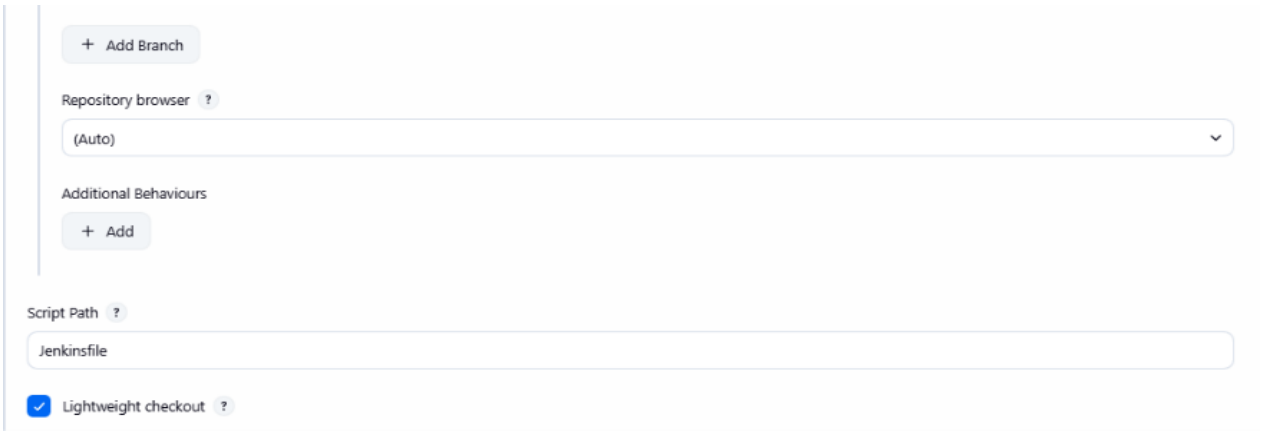
https://github.com/miheeka27/jenkins-demo-app.git

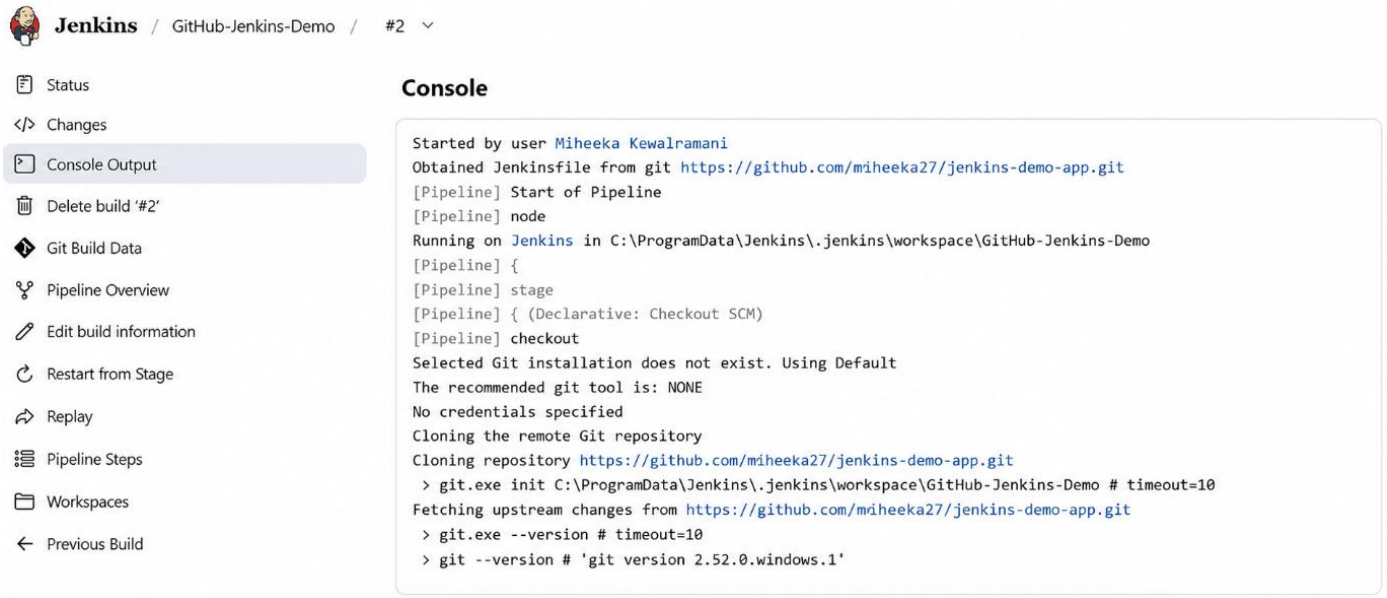
Credentials ?

- none - Add

Branch Specifier (blank for 'any') ?

*/main




```
 > git.exe --version # timeout=10
 > git --version # 'git version 2.52.0.windows.1'
 > git.exe fetch --tags --force --progress -- https://github.com/Miheeka27/jenkins-demo-app.git +refs/heads/*:refs/remotes/origin/*
 # timeout=10
 > git.exe config remote.origin.url https://github.com/Miheeka27/jenkins-demo-app.git # timeout=10
 > git.exe config --add remote.origin.fetch +refs/heads/*:refs/remotes/origin/* # timeout=10
Avoid second fetch
 > git.exe rev-parse "refs/remotes/origin/main"(commit)" # timeout=10
Checking out Revision 0c8d21cde227e66276fd9b2df19fc1815a852884 (refs/remotes/origin/main)
 > git.exe config core.sparsecheckout # timeout=10
 > git.exe checkout -f 0c8d21cde227eb5276fd9b2df19fc1815a852884 # timeout=10
Commit message: "Rename Jenkinsfile.txt to Jenkinsfile"
First time build. Skipping changelog.
[Pipeline]
[Pipeline] // stage
[Pipeline] withEnv
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Build)
[Pipeline] echo
Building the application...
```

jenkins-demo-app / index.html

ascendantcreator Update deployment message and add styling 2a6e95

Code Blame 21 lines (21 loc) • 445 Bytes Raw

```
<IDOCTYPE html>
<html>
<head>
    <title>CI/CD Demo Application</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            text-align: center;
            background-color: #e3f2dd;
            margin-top: 56px;
        )
        Rd {
            color: #0047aj;
        )
    </style>
</head>
<body>
    <h1>Application Updated & Deployed via CI/CD Pipeline!</h1>
    <p>Version 2.0 is live.</p>
</body>
</html>
```

```
Building the application...
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Test)
[Pipeline] echo
Testing the application...
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Deploy)
[Pipeline] echo
Deploying the application...
[Pipeline] bat

C:\ProgramData\Jenkins\.jenkins\workspace\GitHub-Jenkins-Demo>if not exist "C:\Deployments\DemoApp" mkdir "C:\Deployments\DemoApp"
[Pipeline] bat

C:\ProgramData\Jenkins\.jenkins\workspace\GitHub-Jenkins-Demo>copy index.html "C:\Deployments\DemoApp\index.html"
        1 file(s) copied.
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```



**Conclusion:**

Building and executing an automated CI/CD pipeline by connecting Jenkins with GitHub helps automate the process from code development to deployment. Using a **Declarative Pipeline**, code changes can be automatically fetched, tested, and deployed with minimal manual effort.

The main takeaway is the use of **Pipeline as Code** through a **Jenkinsfile**, which keeps the pipeline configuration version-controlled, organized, and reproducible. Overall, the experiment demonstrates how CI/CD automation reduces human errors, speeds up software releases, and enables reliable and consistent software delivery.